

Aplicación de consola: Gestión de estudiantes y notas en JavaScript

1. Descripción de la aplicación

La aplicación permite administrar las notas de un curso. Sus funcionalidades son:

- **Ver estudiantes:** muestra cada estudiante con sus notas.
- **Agregar estudiante:** pide el nombre, la cantidad de notas y cada nota.
- **Ver promedios:** calcula el promedio de cada estudiante.
- **Ver aprobados:** lista los estudiantes cuyo promedio es 4 o más.
- **Ver reprobados:** lista a los demás e indica cuántos puntos les faltan para aprobar.

La lista comienza con cinco estudiantes de ejemplo (Ana, Luis, Camila, Diego y Sofía), con tres notas cada uno. Las notas van de 1 a 7 y se aprueba con un promedio de 4 o más. El promedio se redondea a un decimal. Estos valores están definidos como constantes al inicio del código, por lo que se pueden cambiar fácilmente.

2. Requisitos de la consigna y cómo se cumplen

Requisito	Cómo se cumple
Ejecutarse en la consola del navegador	El código corre en el navegador y escribe sus resultados con console.log.
Ingreso de datos con prompt o variables	Los datos se piden con prompt() y los estudiantes iniciales son datos predefinidos.
Al menos tres operaciones con funciones	sumar, restar y dividir, usadas en el cálculo de promedios y de los puntos que faltan para aprobar, además de las funciones de estudiantes.
Condicionales y repetición	if, else y switch; for y while.
Arreglos y objetos	Arreglo estudiantes con objetos que tienen propiedades, un arreglo notas y métodos.
Recorrido con map o forEach	Las listas se arman con map y se muestran con forEach.
Función que filtra según una condición	filtrarPorAprobacion.
Mensajes con alert	mostrarMensaje y mostrarLineas combinan console.log y alert.

Requisito	Cómo se cumple
Modularización en funciones reutilizables	Funciones pequeñas de una sola responsabilidad (sección 5).

3. Estructura del proyecto

```

proyecto-consola/
├── index.html      # Página básica con botones
├── app.js          # Código fuente (lógica)
├── README.md       # Descripción para el repositorio
├── documentacion.docx # Este documento
└── capturas/       # Capturas de pantalla (se agregan a mano)

```

Archivo	Responsabilidad
index.html	Carga el script y conecta cada botón con una función. No contiene lógica del programa.
app.js	Contiene toda la lógica: entrada de datos, operaciones, estudiantes y menú.
README.md	Presenta el proyecto y explica cómo ejecutarlo.
documentacion.docx	Explica el funcionamiento y justifica las decisiones.
capturas/	Carpeta para las imágenes de la aplicación funcionando, que se usan en el README.

4. Funcionamiento paso a paso

El usuario abre index.html en el navegador y presiona un botón. Cada botón llama a una función de app.js, que pide los datos necesarios con prompt, el cual los valida, ejecuta la operación y muestra el resultado en la consola y en una ventana emergente. A continuación, se explica cada bloque del código.

- **Constantes y datos iniciales**

Se definen NOTA_MINIMA, NOTA_MAXIMA, NOTA_APROBACION y MAX_NOTAS con const para evitar “números mágicos” repartidos por el código. También se crea el arreglo “estudiantes” con cinco objetos de ejemplo, cada uno con su propio arreglo de notas.

- **Entrada y salida**

Las funciones mostrarMensaje, mostrarLineas, pedirNumero, pedirTexto, pedirNota y pedirCantidadNotas tienen una sola responsabilidad cada una. Las de entrada usan un ciclo

while que repite la pregunta hasta recibir un dato válido. Si el usuario cancela, devuelven null y la acción termina sin errores.

En pedirNumero se acepta la coma decimal (por ejemplo, nota 5,5) y se usa Number.isFinite para rechazar valores como abc, NaN o Infinity. Por otro lado, la función mostrarLineas recibe una lista de textos y la imprime con forEach en la consola y con join en una sola alerta.

- **Operaciones matemáticas**

Por otro lado, sumar, restar y dividir son funciones pequeñas que reciben parámetros y retornan un resultado. dividir devuelve null si el divisor es cero.

calcularPromedio recorre el arreglo de notas con un for, acumula la suma con sumar, divide por la cantidad de notas con dividir y redondea a un decimal. calcularPuntosParaAprobar usa restar para saber cuánto le falta a un promedio para llegar a 4 (funciones que llaman a otras funciones).

- **Estudiantes (arreglos y objetos)**

En crearEstudiante se devuelve un objeto con las propiedades nombre y notas y cuatro métodos: promedio(), estaAprobado(), describir() y describirPromedio(). El estado de aprobación no se guarda: se calcula a partir del promedio cada vez que se consulta.

filtrarPorAprobacion recibe la lista y un valor true o false, recorre el arreglo con un for y devuelve solo los estudiantes que cumplen la condición.

- **Acciones del programa**

Las funciones verEstudiantes y verPromedios arman una lista de textos con map y la muestran con mostrarLineas. Por otro lado, agregarEstudiante pide el nombre y la cantidad de notas, y usa un for para pedir cada nota. Si el usuario cancela en cualquier momento, el estudiante no se guarda.

verAprobados y verReprobados filtran la lista con filtrarPorAprobacion. En los reprobados se agrega cuántos puntos faltan para aprobar.

- **Menú principal**

iniciarAplicacion usa un while que muestra el menú hasta que el usuario elige salir (0) o cancela. Un switch llama a la función correspondiente y el default avisa si la opción no es válida.

5. Decisiones del diseño y funcionamiento

- **Uso de index.html con botones.** Además de app.js, el proyecto incluye un index.html mínimo, con un título y un botón por cada función. La decisión se justifica por las siguientes razones:

- **Es necesario para ejecutar el programa.** `prompt()` y `alert()` pertenecen al navegador, por lo que no funcionan con Node ni con el botón de ejecución de un editor. La consigna pide ejecutarlo en el navegador, y para eso se necesita una página HTML que cargue el script.
- **El usuario decide cuándo empezar.** Con un botón, las ventanas de ingreso aparecen al hacer clic y no apenas carga la página, un momento en que algunos navegadores pueden bloquearlas o retrasarlas.
- **Permite probar cada parte por separado.** Un botón por función evita recorrer todo el menú para revisar, por ejemplo, solo la lista de reprobados.
- **Una sola aplicación con un hilo conductor:** la consigna pide implementar operaciones matemáticas y funciones, y exige al menos tres operaciones diferentes. En lugar de una calculadora independiente, las operaciones sumar, restar y dividir se usan dentro de la lógica de notas (promedios y puntos faltantes), de modo que cada operación cumple un propósito real.
- **Tema de estudiantes y notas:** Una lista de estudiantes permite usar de forma natural arreglos, objetos, filtros y promedios.
- **Varias notas por estudiante:** guardar las notas en un arreglo dentro de cada objeto permite trabajar con arreglos de números y recorrerlos con `for`.
- **El estado se calcula, no se guarda:** aprobado o reprobado se obtiene del promedio cada vez que se consulta, por lo que nunca queda desactualizado.
- **Promedio redondeado a un decimal:** el valor que ve el usuario coincide con el que define si aprueba (por ejemplo, un promedio mostrado como 4.0 no aparece como reprobado).
- **La lista general no muestra el estado:** para eso existen los botones de aprobados y reprobados, lo que evita repetir información.
- **Menú con `while` y `switch`:** permite usar varias funciones sin reiniciar el programa y aplica ciclos y condicionales de forma clara.
- **Modo estricto ("`use strict`"):** al inicio de `app.js`, hace que errores como usar una variable no declarada se muestren en la consola en lugar de pasar inadvertidos.
- **Funciones pequeñas:** cada una hace una sola cosa, lo que facilita leerlas y probarlas. Las operaciones simples usan funciones flecha por su brevedad y el resto usa declaraciones de función, que permiten definir los datos iniciales al comienzo del archivo.

- **null para cancelar o para errores esperados:** permite manejar la cancelación del prompt y la división por cero sin lanzar excepciones.
- **Validar en funciones reutilizables:** pedirNumero se reutiliza en pedirNota y pedirCantidadNotas, evitando código duplicado.
- **for y while a propósito:** se recorren los arreglos con for y se usa while para repetir las preguntas y el menú, practicando ambos ciclos.